T10

FICHEROS (II)

TRABAJANDO CON BYTES

- A veces, es posible que necesitemos trabajar con archivos utilizando cadenas de bytes. Esto se hace añadiendo el carácter 'b' al argumento *mode*. Se aplican todos los métodos del objeto *file*. Sin embargo, cada uno de los métodos espera y devuelve un objeto *bytes* en su lugar

- Abrir un archivo de texto utilizando "b" no es interesante. Pero podemos abrir una imagen PNG por ejemplo y examinar el contenido. Como el formato de archivo .png está bien definido, la cabecera del archivo es de 8 bytes.

TRABAJANDO CON BYTES

```
>>> with open('jack_russell.png', 'rb') as
byte_reader:
>>>     print(byte_reader.read(1))
>>>     print(byte_reader.read(3))
>>>     print(byte_reader.read(2))
>>>     print(byte_reader.read(1))
>>>     print(byte_reader.read(1))
b'\x89' # A “magic” number to indicate that
           # this is the start of a PNG
b'PNG'  # PNG in ASCII
b'\r\n' # A DOS style line ending
b'\x1a' # A DOS style EOF character
b'\n'   # A Unix style line ending \n
```

PICKLE

¿QUÉ ES PICKLE?

- Los *pickle* de Python representan un objeto Python como una cadena de bytes. Se pueden hacer multitud de cosas con dichos bytes, como, por ejemplo, almacenarlos en un archivo o base de datos, o transferirlos a través de una red.

- La representación *pickle* de un objeto Python, se denomina archivo *pickle*. El archivo *pickle*, por tanto, se puede utilizar para distintos propósitos, como el almacenamiento de resultados para que sea utilizado por otro programa Python, o bien para crear copias de seguridad. Para obtener el objeto original de Python, simplemente tenemos que hacer *unpickle* a esa cadena de bytes.

¿QUÉ ES PICKLE?

- Para utilizar los *pickle* en Python, utilizaremos el módulo *Pickle*. Este nos permite almacenar casi cualquier objeto Python directamente en un archivo o cadena sin necesidad de realizar ninguna conversión. Lo que el módulo *pickle* realiza en realidad es lo que se llama serialización de objetos, es decir, convertir objetos a y de cadenas de bytes.

- El objeto sobre el que se va a hacer un *pickle* se serializará en un flujo de bytes, los cuales se pueden escribir en un archivo, por ejemplo, y restaurar en un punto posterior.

CREAR UN ARCHIVO PICKLE

```
import pickle

todo = ['write blog post', 'reply to email', \
           'read in a book']
pickle_file = open('todo.pickle', 'wb')
pickle.dump(todo, pickle_file)
pickle_file.close()
```

La función *dump* tiene dos argumentos: el objeto pickle, que es la lista de tareas y un objeto file donde queremos escribir el pickle, que es todo.pickle.

UNPICKLING (RESTAURAR)

```
import pickle


pickle_file = open('todo.pickle', 'rb')
todo = pickle.load(pickle_file)
print(todo)
```

PICKLES COMO STRINGS

```
import pickle

todo = ['write blog post', 'reply to \
          email', 'read in a book’]

pickled_data = pickle.dumps(todo)
print(pickled_data)

restored_data = pickle.loads(pickled_data)
print(restored_data)
```

PICKLES CON MÁS DE UN OBJETO

```
import pickle

name = 'Miguel'
website = 'http://www.materuel.com'
english_french = {'paper': 'papier', \
            'pen': 'stylo', 'car': 'voiture'}
tup = (31, 'abder', 4.0)

pickled_file = open('pickled_file.pickle', 'wb')
pickle.dump(name, pickled_file)
pickle.dump(website, pickled_file)
pickle.dump(english_french, pickled_file)
pickle.dump(tup, pickled_file)
pickled_file.close()
```

PICKLES CON MÁS DE UN OBJETO

```
from pickle import Pickler

name = 'Miguel'
website = 'http://www.miweb.net'
english_french = {'paper': 'papier', \
            'pen': 'stylo', 'car': 'voiture'}
tup = (31, 'abder', 4.0)

pickled_file = open('pickled_file.pickle', 'wb')
p = Pickler(pickled_file)
p.dump(name)
p.dump(website)
p.dump(english_french)
p.dump(tup)
pickled_file.close()
```

PICKLES CON MÁS DE UN OBJETO

```
import pickle

pickled_file = open('pickled_file.pickle', 'rb')

name = pickle.load(pickled_file)
website = pickle.load(pickled_file)
english_french = pickle.load(pickled_file)
tup = pickle.load(pickled_file)

print('Name: ')
print(name)
print('Website:')
print(website)
print('English to French:')
print(english_french)
print('Tuple data:')
print(tup)
```

PICKLES CON MÁS DE UN OBJETO

```
from pickle import Unpickler

pickled_file = open('pickled_file.pickle', 'rb’)

u = Unpickler(pickled_file)

name = u.load()
website = u.load()
english_french = u.load()
tup = u.load()

print('Name: ')
print(name)
print('Website:')
print(website)
print('English to French:')
print(english_french)
print('Tuple data:')
print(tup)
```

PICKLES CON MÁS DE UN OBJETO

```
import pickle

name = 'Miguel'
website = 'http://www.miweb.net'
english_french = {'paper': 'papier', \
             'pen': 'stylo', 'car': 'voiture'}
tup = (31, 'abder', 4.0)

pickled_file = open('pickled_file.pickle', 'wb')

data = {'name': name, 'website': website, \
             'english_french_dictionary':english_french,\
             'tuple': tup}

pickle.dump(data, pickled_file)
pickled_file.close()
```

Las variables deben escribirse y leerse en el mismo orden para obtener la salida deseada. Para evitar cualquier problema, podemos utilizar un diccionario para administrar los datos

PICKLES CON MÁS DE UN OBJETO

```
import pickle

pickled_file = open('pickled_file.pickle', 'rb')
data = pickle.load(pickled_file)

name = data['name']
website = data['website']
english_french = data['english_french_dictionary']
tup = data['tuple']

print('Name: ')
print(name)
print('Website:')
print(website)
print('English to French:')
print(english_french)
print('Tuple data:')
print(tup)
```

Las variables deben escribirse y leerse en el mismo orden para obtener la salida deseada. Para evitar cualquier problema, podemos utilizar un diccionario para administrar los datos

EJERCICIO 10.1

JSON

¿QUÉ ES JSON?

- Para responder qué es JSON, debemos empezar por decir que sus siglas en inglés son por *JavaScript Object Notation*.

- Se trata de un formato para guardar e intercambiar información que cualquier persona pueda leer. Los archivos json contienen solo texto y usan la extensión .json.

- JSON es un formato que almacena información estructurada y se utiliza principalmente para transferir datos entre un servidor y un cliente.

¿QUÉ ES JSON?

- Hay dos elementos centrales en un objeto JSON: claves (Keys) y valores (Values).
    - Las Keys deben ser cadenas de caracteres (strings). Como su nombre en español lo indica, estas contienen una secuencia de caracteres rodeados de comillas.
    - Los Values son un tipo de datos JSON válido. Puede tener la forma de un vector (array), objeto, cadena (string), booleano, número o nulo.
- Un objeto JSON comienza y termina con llaves {}. Puede tener dos o más pares de claves/valor dentro, con una coma para separarlos. Así mismo, cada *key* es seguida por dos puntos para distinguirla del valor.
- Como resultado, la sintaxis literal de un objeto JSON es así:

```
{“key”:“value”,“key”:“value”,“key”:“value”}
```

SERIALIZACIÓN EN JSON

- La librería *json* expone el método *dump()* para escribir datos en archivos. También existe un método *dumps()* para escribir en una cadena.

- Los objetos Python simples se traducen a JSON según una conversión bastante intuitiva:

Python	JSON (JavaScript)
dict	object
list, tuple	array
str	string
int, long, float	number
True	true
False	false
None	null

SERIALIZACIÓN EN JSON

```
import json

data = {
    "president": {
        "name": "Zaphod Beeblebrox",
        "species": "Betelgeusian"
    }
}

with open("data_file.json", "w") as write_file:
    json.dump(data, write_file)

json_string = json.dumps(data)

print(json_string)
```

SERIALIZACIÓN EN JSON

```
import json

data = {
    "president": {
        "name": "Zaphod Beeblebrox",
        "species": "Betelgeusian"
    }
}

with open("data_file.json", "w") as write_file:
    json.dump(data, write_file, indent=4)

json_string = json.dumps(data, indent=4)

print(json_string)
```

DESERIALIZACIÓN EN JSON

- En la librería *json*, encontramos *load()* y *loads()* para convertir datos codificados en JSON en objetos Python.

- Al igual que la serialización, hay una tabla de conversión simple para la deserialización:

JSON	Python
object	dict
array	list
string	str
number (int)	int
number (real)	float
true	True
false	False
null	None

DESERIALIZACIÓN EN JSON

```
import json

with open("data_file.json", "r") as read_file:
    data = json.load(read_file)

print(data)

json_string = """
{
    "researcher": {
        "name": "Ford Prefect",
        "species": "Betelgeusian",
        "relatives": [
            {
                "name": "Zaphod Beeblebrox",
                "species": "Betelgeusian"
            }
        ]
    }
}
"""
data = json.loads(json_string)

print(data)
```

EJERCICIO 10.2

MANEJO DE FICHEROS

COMPROBAR SI UN FICHERO EXISTE

```
>>> import os

>>> existe = os.path.exists(fichero)
```

LISTADO DE DIRECTORIOS

```
>>> import os

>>> entries = os.listdir('my_directory/')
```

LISTADO DE DIRECTORIOS

```
import os

with os.scandir('my_directory/') as entries:
    for entry in entries:
        print(entry.name)
```

LISTADO DE FICHEROS EN DIRECTORIOS

```
import os

# List all files in a directory using scandir()

basepath = 'my_directory/’

with os.scandir(basepath) as entries:
    for entry in entries:
        if entry.is_file():
          print(entry.name)
          print(entry.path)
```

LISTADO DE SUBDIRECTORIOS

```
import os

# List all subdirectories using scandir()

basepath = 'my_directory/’

with os.scandir(basepath) as entries:
    for entry in entries:
        if entry.is_dir():
            print(entry.name)
```

ATRIBUTOS DE FICHEROS

```
>>> import os

>>> with os.scandir('my_directory/') as
dir_contents:
...     for entry in dir_contents:
...         info = entry.stat()
...         print(info.st_mtime)
...

1539032199.0052035
1539032469.6324475
1538998552.2402923
1540233322.4009316
1537192240.0497339
1540266380.3434134
```

CREAR UN DIRECTORIO

```
import os

try:
    os.mkdir('example_directory/')
except FileExistsError:
    print('El directorio ya existe')
```

CREAR VARIOS DIRECTORIOS

```
import os

os.makedirs('2018/10/05')
```

```
.
└── 2018/
    └── 10/
        └── 05/
```

ELIMINAR FICHEROS

```
import os

data_file = 'home/data.txt'

# Use exception handling

try:
    os.remove(data_file)
except OSError as e:
    print(f'Error: {data_file} : {e.strerror}')
```

ELIMINAR DIRECTORIOS

```
import os

trash_dir = 'my_documents/bad_dir'

try:
    os.rmdir(trash_dir)
except OSError as e:
    print(f'Error: {trash_dir} : {e.strerror}')


import shutil

shutil.rmtree('/folder_name', ignore_errors=True)
```

COPIAR FICHEROS

```
import shutil

src = 'path/to/file.txt’

dst = 'path/to/dest_dir’

shutil.copy(src, dst)

# Conservadndo metadatos

# shutil.copy2(src, dst)
```

COPIAR DIRECTORIOS

```
>>> import shutil

>>> shutil.copytree('data_1', 'data1_backup')
'data1_backup'
```

MOVER FICHEROS Y DIRECTORIOS

```
>>> import shutil

>>> shutil.move('dir_1/', 'backup/')
'backup'
```

RENOMBRAR FICHEROS Y DIRECTORIOS

```
>>> os.rename('first.zip', 'first_01.zip')
```

EJERCICIO 10.3

EJERCICIO 10.4